

Velocity — Customer Context Layer

The layer that gives each AI agent exactly the context its job needs — across many brands

Short version (10 slides). Full reasoning in DECK.md.

Slide 1 · Thesis

I didn't build one agent. I built the layer that makes *all* the agents possible.

A **CDP** answers "*who is my customer?*" But an agent about to act doesn't need a 360° profile — it needs **the few facts that change its decision, right now.**

We built the layer that answers: "**what does THIS agent need to know about THIS customer to do THIS job, at the moment it acts?**" — and across many brands, the data compounds into a moat no single brand can build.

Slide 2 · The problem

Velocity's customer data is scattered across storefront, payments, courier, WhatsApp, web sessions, marketing — and the same person shows up as several different records.

There are ~10 products on the roadmap (WISMO, COD→prepaid, RTO prevention, cart recovery, winback, upsell...). **Every one fails the same way: it acts on partial context.**

The bottleneck isn't the agents. It's the context they stand on. Fix that once, and every agent — today's and the ones not built yet — gets better at the same time.

Slide 3 · The reframe (+ my method)

	CDP / "profile-first"	Context Layer / "job-first" (ours)
Question	Who is this customer?	What does this agent need to act?
Output	One big profile	A tight, ranked bundle per task
New product	Re-query, re-decide what matters	Declare a Context Contract
Cost at the model	Whole profile every prompt	Only the job's fields (~70-90% smaller)

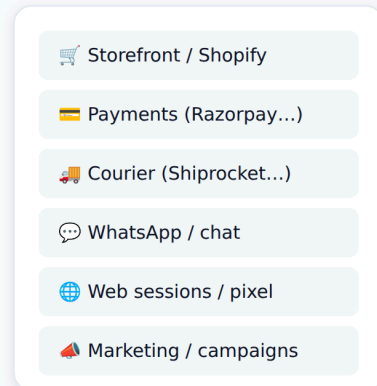
The method: I didn't design the data model in a vacuum. I let the **agents' needs be the forcing function** — each agent declares what it needs to act, and that defines what the layer must serve. *That's why it serves products we haven't built yet.*

Slide 4 · Architecture (one picture)

Velocity · Customer Context Layer — Architecture

The agents' needs are the forcing function: each agent declares a Context Contract; the layer is built to serve exactly those needs — across many brands, one platform.

SOURCES · per brand × N brands



ingest

PLATFORM · one, multi-tenant

1 Ingest & AI schema mapping
LLM maps any source's raw fields → canonical events.
AI replaces hand-written per-brand ETL → onboard in hours.

2 Identity resolution (the keystone)
Fragments → one human, with a confidence on every link:
• **deterministic** phone/email · • **probabilistic** name+addr+device
• **behavioral** session→buyer · • **cross-brand** graph join ★

3 · UNIFIED STORE (split on purpose)

3a · Event store

Immutable, append-only
log: orders, chats,
sessions, shipments.

Source of truth · replayable

3b · Trait store

Derived, recomputed
from events: RFM, LTV,
RTO-risk, churn, sentiment.

AI-extracted · never hand-set

A wrong trait? Replay events & recompute → no mystery state. Agents

4 Context API · getContext(customer, task)

contract → resolve fields → rank by relevance + recency → bundle

◀ **THIS IS THE PROTOTYPE** ▶

tight, task-scoped context — ~70-90% smaller than a full profile dump

AGENTS · each = one Context Contract

COD → Prepaid

cod_ratio · rto_risk · aov · ltv

RTO Shield ★ cross-brand moat

rto_risk · cross_brand_return_signal

Cart Recovery

cart · sentiment · pref_payment

Winback

days_since · churn · fav_category

Proactive WIMO

shipment · anxiety · is_first_order

Upsell + Review

last_order · sentiment · complements

Product #N (not built yet)

just declare a new contract →

Why it scales

New agent = new contract over
existing traits. No re-architecture.

Platform cost ≈ O(1) in #products.

The roadmap speeds up as it grows.

CLOSED LOOP · the layer learns

Every action & outcome (converted? returned? replied?) is written back as a new **event** →

traits like **past_nudge_response** and **rto_risk** sharpen → the **next** decision is better. Self-tuning, no analyst in the loop.

Cross-brand: every new brand improves the network RTO/churn priors for every other brand — the data compounds into a moat no single brand can build.

Sources → ① **AI schema mapping** (any source → canonical events) → ② **identity resolution** (fragments → one human, with confidence) → ③ **event store** (immutable truth) + **trait store** (derived, recomputed) → ④ **Context API** `getContext(customer, task)` (*this is the prototype*) → **agents**, each one contract → **closed loop** (outcomes sharpen the next decision).

Slide 5 · #1 Unify — identity + two stores

Identity resolution (the keystone). Fragments → one human, with a **confidence on every link**: deterministic (phone/email) → probabilistic (name+address+device, AI) → behavioral (session→buyer) → cross-brand graph join. We tune for **precision over recall** — a wrong merge is worse than a missed one. *Prototype: Priya's 4 records → 1.*

Two stores, on purpose. Event store = immutable facts (replayable truth). Trait store = derived signals (RFM, RTO-risk, churn, sentiment), **recomputed from events, never hand-set**. A wrong trait? Replay and recompute — and because agents read *traits*, recomputation never breaks a contract.

Slide 6 · #2 AI-first — AI does the work, not rules/humans

Where	Today (manual)	With AI
Ingestion	Hand-map each brand's schema	LLM maps any schema → canonical events
Identity	Exact-match rules only	Probabilistic matcher with confidence
Traits	Analysts write SQL segments	Model extracts sentiment, intent, RTO-risk
Segmentation	Marketer waits on data team	Plain English → live segment <i>(shown)</i>
Action	Static templated copy	Per-customer message from the bundle <i>(shown)</i>

The prototype demonstrates the last two **live**: type an audience in English → AI builds the segment; minimal context → Claude writes the WhatsApp message.

Slide 7 · #3 The cross-brand moat

A single brand sees **Vikram** place one COD order and return it — annoying, not a pattern.

Velocity sees 4 of 5 returned across three brands. So the next COD order — at a brand he's never bought from — gets COD paused and prepay required *before we ship at a loss*.

No single brand can build this. It only exists because we're one platform serving many brands — and every brand that joins improves the RTO/churn priors for every other brand. **The data layer isn't a cost center; it's the network effect.**

Slide 8 · #4 Scale — the Context Contract is the whole trick

New product on the roadmap? → Write a contract. → Ship.
(not: design a schema, build a pipeline, re-resolve identity)

- A contract is a small declarative list of trait/event keys + decision logic.
- It reads the **existing** trait store — usually needs zero new data.
- A new trait, computed once, is then available to **every future** agent.

Result: platform cost $\approx O(1)$ in number of products. Multi-tenant by default; cross-brand sharing is governed (derived scores, never raw PII). *The roadmap speeds up as it grows.*

Slide 9 · #5 Build first vs defer — and why

Prioritized by **value × data-readiness × reversibility-of-risk**.

Build first: Foundation (identity + stores + Context API) → then low-regret, clean-data agents: **Proactive WIMO, Cart Recovery, COD→Prepaid** (strong signal, reversible, immediate value).

Defer (and why): **RTO Shield** (needs volume — pausing COD on a good customer is a costly false positive) · **Winback** (needs more history) · **autonomous send, broad broadcasts, returns adjudication, ML retraining infra** (high blast radius or money/policy — earn trust first).

The restraint *is* the answer: the prototype has no DB, no auth, no real integrations — on purpose. Knowing what not to build is the same judgment this question tests.

Slide 10 · The one line

Today's WhatsApp agent gets smarter the day this layer ships. Every future agent just declares a contract over the same data. And because it's one platform across many brands, the data compounds into a moat no single brand can build.

Why it's the right answer, not just an answer: fixes the shared bottleneck (not one feature) · AI-first and proven live · serves products not built yet · the restraint signals seniority.

The prototype makes it clickable. This deck is how it scales.